FINAL PROJECT · DATA ENGINEERING

FunnelStream

Real-time e-commerce and campaign analytics, from Kafka to Power BI

Design documentation v4 · 4 October 2026 · Defense 13 October 2026

1. Objective and problem

Modern e-commerce and marketing teams generate data in many systems: app and web events, orders, delivery, Meta and Google Ads. The sources differ in format and speed. Teams learn campaign results late, cannot trace a sale to the ad that caused it, and do not see where cash-on-delivery (COD) orders are lost.

Main objective: show which campaigns turn ad spend into delivered orders, where shoppers drop off from product view to delivery, and where COD losses come from.

Business questions

- Spend, CPA and ROAS per campaign, per day. Headline ROAS uses delivered revenue; placed-order ROAS is secondary.
- Conversion rate and cart and checkout abandonment per channel and platform, split by payment method (COD vs prepaid).
- Funnel from product view to cart, checkout, order and delivery, with drop-off at each step.
- Cancellation rate, failed-delivery rate and delivery lead time, per carrier and city.
- Products browsed and bought per city, and which products to recommend next (labelled co-view).

2. Scope

Required (must ship)	Stretch (cut first if late)	Out of scope
Python simulator: app, web, orders, delivery, catalog export	Customer 360 table and Power BI page	Grafana, Prometheus, live operational alerts
Mock Meta and Google Ads API, on-demand pulls	10 million event benchmark (one-off)	Schema Registry, Avro, Protobuf
Kafka: app.events, orders, delivery.events	Contribution margin after COD and cancellation cost	Cloud deployment
Spark Structured Streaming: Bronze, Silver, Gold	Anomaly alert rules	ALS collaborative filtering
Airflow DAG (manual trigger) and Python fallback script		Real-time recommendations in the web app
Delta Lake, Medallion layers: Bronze and Silver local, Gold in Databricks		Incrementality tests, view-through, CAC payback
Databricks SQL serving Gold (Postgres fallback); Power BI (Import mode)		Real Meta or Google accounts; refunds and partial cancels
Metric dictionary, SQL data-quality checks, sample CSVs		Great Expectations and dbt (optional if time allows)

Volume: the pipeline must run end to end at 1–2 million events. 10 million events is a one-off scale benchmark, recorded with timings.

3. Architecture

Stream path (live)

Simulator → Kafka (app.events, orders, delivery.events) → Spark bronze_ingest → Bronze Delta → Spark silver_clean → Silver Delta (both on local disk). The local job also writes live_metrics_minute.

Publish to Databricks

Databricks cannot reach the laptop's Kafka, so an export step uploads the Silver tables to a Databricks volume as Parquet and loads them into Silver mirror tables. It runs on demand and after each stream session.

Batch path (manual Airflow trigger)

Ad API pull → Bronze campaign snapshots (pulled_at) → Silver → Databricks Gold. Catalog and customer export → Bronze → Silver (with updated_at) → SCD2 dimensions in Databricks. Gold facts, attribution, recommendations and city-product tables are built in Databricks from the Silver mirror. If Airflow is not stable by 11 October, the same steps run as a Python script (scripts/run_batch.py). If Databricks is not usable by 10 October, the same Gold tables load into local Postgres through the same loader interface and Power BI reads Postgres.

Single writer per Gold table

Owner	Tables
Local Spark stream	Bronze, Silver, live_metrics_minute (local Delta)
Databricks Gold build	every Gold table: dim_customer, dim_product, dim_campaign, dim_date, fact_orders, fact_order_lines, fact_funnel_daily, fact_attribution, fact_campaign_daily, fact_product_recs, fact_city_product_daily, fact_customer_360 (stretch)

Processing guarantee

At-least-once delivery with idempotent writes. Silver deduplicates on event_id and uses Delta MERGE. Gold upserts on its primary key. Results are exactly-once in effect; the system does not claim a broker switch for it.

Surrogate keys

customer_key, product_key and campaign_key are resolved at load time from the dimension version valid at event time. Unmatched rows go to quarantine.

Storage and serving

Bronze and Silver are Delta folders under data/lake. Spark checkpoints live under data/checkpoints. There is no object store. Gold is Delta in Databricks (funnelstream.gold). Power BI uses the Databricks connector in Import mode and refreshes after each batch run. Credentials sit in .env. Code depends on one loader interface (GoldStore) with a Databricks backend and a Postgres backend.

4. Data sources, clicks and realism

Source	Mode	Delivery	Events / data
Mobile app and website	Streaming	Kafka, keyed by session_id	app_open, page_view, product_view, product_click, product_view_end, product_search, add_to_cart, remove_from_cart, checkout_started, purchase
Orders	Streaming	Kafka, keyed by order_id	order_created, payment_authorised, order_packed, order_shipped, order_cancelled, order_completed
Delivery	Streaming	Kafka, keyed by order_id	out_for_delivery, delivered, delivery_failed
Catalog and customers	Batch, daily	CSV export, Airflow load	Products (price, category) and customers (city, primary_device) with updated_at for SCD2
Meta and Google Ads (mock)	Batch, on demand	Airflow pulls mock API	Impressions, clicks and spend per campaign, cumulative for the day

Click definition

A session click is the first page_view or product_view in a session that carries utm_campaign. Attribution uses this session click (click_id). CTR uses clicks reported by the ad API. The dashboard labels each one, because the counts differ.

Egyptian realism

- Names: Egyptian first and last names. Names and phones are not kept in Gold.
- Cities weighted by population: Cairo, Giza, Alexandria, Mansoura, Tanta, Assiut and others.
- Phones: +20 followed by 10, 11, 12 or 15, then 8 digits (Bronze and Silver only).
- Currency: EGP.
- Payments: cash on delivery as the largest share, then Fawry, Vodafone Cash, InstaPay and cards.
- Carriers: Bosta, Aramex, J&T; Express. Failure rates differ by city.
- Catalog: fashion, beauty, electronics, home.
- Seasonality: Ramadan, Eid, Friday and Saturday peaks, evening traffic. Times stored in UTC; event_date in Africa/Cairo.

Limit: synthetic data cannot be proven 100% realistic. The team documents these assumptions and checks output distributions against them.

Injected noise and how it is handled

The simulator injects seeded noise at configurable rates and writes noise_manifest.json with exact counts. The SQL checks compare the Silver quarantine rate and duplicate count with that manifest; they must match within a small tolerance, which proves the quality layer works.

Noise	Rate	Handled by
Duplicate event	1%	Silver dedup on event_id
Malformed JSON	0.3%	Quarantine (BAD_JSON)
Missing required field	0.3%	Quarantine (MISSING_FIELD)
Unknown event type or version	0.2%	Quarantine (UNKNOWN_EVENT_TYPE, BAD_VERSION)

Noise	Rate	Handled by
Negative or zero price; non-numeric number	0.2%	Quarantine (NEGATIVE_PRICE, BAD_VALUE)
Future timestamp	0.1%	Quarantine (FUTURE_TIMESTAMP)
Late (10-120 min) and very late (days)	2% and 0.3%	is_late flag; excluded from live metrics; batch corrects
Non-UTC timestamp offset	1%	Normalised to UTC
Null or unknown product_id	0.5% and 0.3%	Quarantine (MISSING_FIELD, UNKNOWN_PRODUCT)
Dirty city or device text	3%	Standardised with a lookup (contracts/normalize.py)
Number sent as string	0.3%	Cast to decimal; quarantine if it fails
Out-of-order status; cancel after shipment	0.5% and 0.3%	Status rank; cancel to quarantine
Bot sessions	0.1% of sessions	is_bot flag; excluded from funnel and attribution
Ad API: duplicate pull, clicks above impressions, negative spend, HTTP 503	small	Latest-pull rule, SQL checks, retries
Catalog export: duplicate rows, null city	small	Dedup on key; default unknown

5. Kafka and order lifecycle

Topic	Key	Partitions	Retention	Purpose
app.events	session_id	6	3 days	Clickstream; anonymous events have no customer_id
orders	order_id	3	7 days	Order status changes
delivery.events	order_id	3	7 days	Delivery steps and failures

Producers use acks=all and idempotence. Kafka is a buffer for replay; Bronze is the record.

order_created payload

order_id, customer_id, session_id, channel, utm_source, utm_campaign, click_id, payment_method, city, country, items (product_id, quantity, unit_price, line_total), order_total.

Main path and status rank

order_created → payment_authorised → order_packed → order_shipped → out_for_delivery → delivered → order_completed. Current status is the highest rank, not the latest arrival.

Status	Rank	Status	Rank
order_created	1	out_for_delivery / delivery_failed	5
payment_authorised	2	delivered	6
order_packed	3	order_completed	7
order_shipped	4	order_cancelled	valid only before rank 4; terminal

Branches: delivery_failed returns to out_for_delivery and increments delivery_failed_count. A cancel after order_shipped goes to quarantine.

6. Medallion layers and data dictionary

All layers are Delta tables: Bronze and Silver on local disk, Gold in Databricks (Silver is mirrored there). Bronze never changes data. Silver fixes it. Gold answers questions.

Bronze (raw, append only, kept forever)

Table	Source	Columns
bronze_app_events	app.events	raw_value, kafka_partition, kafka_offset, kafka_timestamp, ingest_time, ingest_date
bronze_orders	orders	Same as bronze_app_events
bronze_delivery_events	delivery.events	Same as bronze_app_events
bronze_campaign_snapshots	Ad API pull	raw_json, pulled_at, platform, ingest_date
bronze_customers, bronze_products	Catalog export	raw_row, file_name, ingest_date

Silver (validated, typed, deduplicated)

Table (key)	Columns
silver_events (event_id)	event_type, event_time (UTC), kafka_timestamp, is_late, event_version, session_id, customer_id (nullable), device, channel, country, city, campaign_id, utm_source, utm_campaign, click_id, product_id, element, dwell_seconds, cart_id, order_id, search_term, event_date (Africa/Cairo)
silver_order_status (event_id)	order_id, customer_id, status, status_rank, status_time, payment_method, carrier, reason, event_date
silver_order_items (order_id, line_no)	product_id, quantity, unit_price (EGP), line_total (EGP), currency, order_time
silver_campaign_snapshots (campaign_id, report_date, pulled_at)	utm_campaign, platform, impressions, clicks, spend (EGP), currency
silver_customers (customer_id, updated_at)	country, city, signup_date, primary_device, marketing_consent
silver_products (product_id, updated_at)	name, category, price (EGP), is_active
silver_quarantine	raw_payload, source_topic, error_code, error_detail, received_at

Gold (business-ready, built and served in Databricks)

Table (key)	Owner	Columns
dim_customer (customer_key, SCD2)	Airflow	customer_id, country, city (tracked), signup_date, primary_device (tracked), marketing_consent, valid_from, valid_to, is_current
dim_product (product_key, SCD2)	Airflow	product_id, name, category (tracked), price (tracked), is_active, valid_from, valid_to, is_current
dim_campaign (campaign_key)	Airflow	campaign_id, utm_campaign, platform, name, start_date, end_date
dim_date (date_key)	Airflow	date, week, month, is_weekend, is_friday_saturday, is_ramadan, is_eid, is_holiday
fact_orders (order_id)	Spark	customer_key, session_id, channel, utm_source, utm_campaign, click_id, campaign_id, payment_method, carrier, city, country, order_total (EGP), current_status, status_rank, order_created_at, delivered_at, cancelled_at, delivery_lead_hours, delivery_failed_count, first_attempt_delivered, is_cancelled, is_delivered, order_seq, is_first_order, date_key
fact_order_lines (order_id, line_no)	Spark	customer_key, product_key, date_key, city, quantity, line_total (EGP)
fact_funnel_daily (event_date, country, channel, payment_method)	Spark	sessions, product_views, cart_sessions, checkout_sessions, orders, cart_abandon_rate, checkout_abandon_rate
live_metrics_minute (minute_start, country)	Spark	events, active_sessions, product_views, add_to_cart, orders_placed (1-minute windows, non-late rows)
fact_attribution (order_id, model)	Airflow	campaign_id (NULL = direct), click_id, order_total (EGP), model (last_touch, first_touch)
fact_campaign_daily (campaign_id, report_date)	Airflow	platform, impressions, clicks, spend (EGP), orders_placed, orders_delivered, revenue_placed (EGP), revenue_delivered (EGP), orders_cancelled, orders_failed_delivery, cancel_rate, failed_delivery_rate, new_customer_orders, aov (EGP), ctr, conversion_rate, cpa (EGP), roas_placed, roas_delivered, direct_share
fact_product_recs (segment_level, segment_value, anchor_product_id, rank)	Airflow	rec_product_id, co_view_count, window_end
fact_city_product_daily (city, product_id, date_key)	Airflow	sessions_viewed, add_to_cart, units, revenue (EGP)

Stretch: cfg_cost_assumptions (margin_pct, carrier_fee_egp, failed_attempt_cost_egp, cod_fee_egp) adds contribution_egp to fact_orders and fact_campaign_daily. Labelled assumption-based.

7. Data modeling: Kimball, galaxy schema, SCD2

Method: Kimball star schemas in Gold. Silver is the integrated, normalized layer, which gives Inmon-style integration without a full enterprise warehouse.

Galaxy schema: several fact tables share the same dimensions (dim_customer, dim_product, dim_campaign, dim_date). This is also called a fact constellation.

SCD Type 2: applies to dim_customer (city, primary_device) and dim_product (price, category). valid_from comes from updated_at in Silver. Facts join to the version valid at event time (valid_from ≤ event time < valid_to). dim_campaign and dim_date have no history.

Attribution: last click within 7 days (last_touch) and first click within 7 days (first_touch). Direct orders are one row with NULL campaign and click.

Recommendations: co-view over 30 days. Top 5 per segment: city, then country, then global when a city has fewer than 20 co-viewing customers. Only active products. Labelled co-view, not purchase.

Headline KPIs:

```
ROAS (delivered) = revenue_delivered / spend

ROAS (placed) = revenue_placed / spend

CPA = spend / orders_placed

Cancel rate = orders_cancelled / orders_placed

Failed-delivery rate = orders_failed_delivery / orders_placed
```

Full formulas: docs/metrics.md.

8. Power BI pages

Page	Content
1. Executive	Delivered revenue, placed revenue, AOV, ROAS (delivered), trend against last week
2. Campaign	Spend, CPA, ROAS placed and delivered, CTR, conversion rate by campaign and platform. Platform-reported conversions beside attributed ones. Shows the 2–4 day delivered-revenue lag. Headline: a cheap-looking campaign can be the least profitable after COD failures.
3. Funnel	Views, cart, checkout, orders, delivered, abandonment rates. Split by channel and payment method
4. Delivery and COD	Cancel rate, failed-delivery rate, first-attempt rate, lead time by carrier and city
5. City and product	Top products per city; co-view recommendations (labelled co-view)
6. Customer 360 (stretch)	Segment mix, repeat-buyer rate, drill-through to one customer

Power BI shows customer_id only. No names or phone numbers.

9. Customer 360 view (stretch)

One row per customer, rebuilt in each batch run. Built last. It is cut first if the schedule slips. The new-vs-repeat and AOV metrics in fact_campaign_daily cover most of the need without it.

Column group	Columns in fact_customer_360 (key: customer_id)
Identity	customer_key, country, city
Browsing	total_sessions, product_views, add_to_cart_count, checkouts_count
Orders	orders_count, delivered_count, cancelled_count, first_order_at, last_order_at, days_since_last_order
Value	total_spend (EGP), avg_order_value (EGP), is_repeat_buyer
Preferences	top_category, preferred_payment_method, last_seen_at
Marketing	last_campaign_id

10. Data quality and reliability

SQL checks run after every load. A failed check stops the Gold load.

Table	Check	Threshold
All tables	Duplicate primary key	0
silver_events	Null event_id or event_time; event_time in the future	0
silver_events	is_late rate	Report; alert above 5%
silver_events	product_id not in silver_products	0
silver_order_status	Status not in allowed list; cancel after order_shipped	0 (cancel goes to quarantine)
silver_order_items	line_total ≠ quantity × unit_price	0
fact_orders	order_total ≠ sum of fact_order_lines.line_total	0
silver_campaign_snapshots	clicks > impressions; negative spend	0
dim_customer, dim_product	Overlapping validity; not exactly one is_current per key	0
Facts → dimensions	Orphan keys	0
Silver → Gold	Row-count reconciliation (orders, lines)	Match
silver_quarantine	Share of all rows	Report; alert above 2%

Reliability

- Kafka to Spark: checkpoints on local disk record committed offsets. A restarted job resumes without losing rows.
- Duplicates removed in Silver by event_id or natural key. MERGE makes replays idempotent. Gold upserts on its key.
- Late events: 10-minute watermark for live metrics. Late rows kept in Silver; the batch run corrects Gold.
- Out-of-order status resolved by status rank. Bad messages quarantined with error code and reason.
- Airflow tasks retry 2 times with exponential backoff. Silver and Gold can be rebuilt from Bronze.

Security

- Secrets in .env, never in Git. .env.example lists the names.
- Databricks: Power BI uses a read-only token limited to funnelstream.gold; the write token is used only by the load job. Tokens live in .env. Fallback Postgres roles: analyst, engineer, admin.
- Gold holds no names or phone numbers. Synthetic data only.

11. Success criteria, risks and timeline

Success criteria

- Stack starts with one command on each laptop, within the Compose memory limits.
- Pipeline runs end to end at 1–2 million events; one 10 million benchmark recorded with timings.
- A campaign DAG run shows numbers up to the trigger time.
- Power BI shows the five business questions, with delivered ROAS as the headline.
- Failure demo: Spark job killed and restarted, no duplicate rows in Gold.
- docs/metrics.md and SQL checks committed; sample CSVs for every Silver and Gold table.

Risks

Risk	Mitigation
9 days to deadline	Cut order: Customer 360, contribution margin, 10M benchmark, dbt, Great Expectations
Laptop memory: Kafka, Spark, Postgres, Airflow	Compose memory limits; run 1–2M events; Python fallback for batch
Databricks access, limits or upload time	Create the workspace on 5 Oct, test one upload on 6 Oct, fall back to Postgres by 10 Oct
SCD2 history and joins	Test one product price change; SQL overlap checks
Synthetic realism questioned	Document assumptions and distributions
Delivered revenue lag	Show the lag on the dashboard

Timeline

Dates	Work
4–5 Oct	Setup, Docker Compose with memory limits, simulator and catalog export
6–7 Oct	Kafka topics, Spark Bronze and Silver with status rank
8–9 Oct	Databricks Gold build, Silver publish, attribution (last and first touch)
10–11 Oct	Airflow batch DAG (or Python fallback), Power BI pages 1–5
11–12 Oct	Customer 360 (stretch), metrics.md and SQL checks
12 Oct	Failure demo, 1–2M run, rehearsal
13 Oct	Defense

12. Tech stack

Layer	Technology	Used for
Language	Python 3	Simulator, mock ad API, catalog export, Airflow DAGs, PySpark jobs, batch fallback script
Query	SQL (Databricks SQL, ANSI)	Attribution, KPIs, data-quality checks
Event streaming	Apache Kafka (KRaft)	app.events, orders, delivery.events; dead-letter topics
Processing	Apache Spark, PySpark Structured Streaming	bronze_ingest, silver_clean, live_metrics_minute, order and funnel Gold tables
Table format	Delta Lake	ACID writes, MERGE for dedup, time travel for replay
Cloud lakehouse	Databricks (Free Edition or trial), Unity Catalog	Silver mirror, Gold tables, SQL warehouse
Orchestration	Apache Airflow	Batch DAG on manual trigger, retries with backoff
Fallback serving	PostgreSQL (Docker)	Same Gold tables behind the same loader if Databricks is unusable
Dashboards	Power BI Desktop	Databricks connector, Import mode; six pages
Containers	Docker, Docker Compose	One command starts the stack, with memory limits
Ad data	Mock Meta and Google Ads API	Campaign snapshots with pulled_at

Optional, cut first if behind: dbt, Great Expectations. Not in scope: Grafana, Prometheus, other cloud services.